

APPENDIX A ARTIFACT APPENDIX

A. Description & Requirements

1) *How to access*: Our code can be pulled from the public repository on GitHub. Use the following command: `git clone https://github.com/XiaochenLi-w/Stream-release-with-delay-time.git`.

2) *Hardware dependencies*: None.

3) *Software dependencies*: Python 3.10 and above. ‘numpy’, ‘math’, ‘random’, ‘os’, ‘sys’, ‘matplotlib’, and ‘scikit-learn’ libraries are required.

4) *Benchmarks*: All datasets used in the experiments are stored in the ‘./data’ folder.

B. Artifact Installation & Configuration

We have tested the source code in Python 3.10.8 and 3.11.4 environments, the code runs without issues in these environments. Additionally, please ensure that the ‘numpy’, ‘math’, ‘random’, ‘os’, ‘sys’ and ‘matplotlib’ libraries are installed. The code does not have high hardware requirements; a single-core CPU is sufficient.

C. Major Claims

- (C1): Allowing a delay of only 10 timestamps can bring about a substantial increase in accuracy for the released streams (Compare CompOrder, Discontin_pp, BucOrder, Contin_reduce, Discontin_reduce with Naive, PeGaSus, PeGaSus_delay).
- (C2): The performance of PeGaSus is not improved compared to the naïve approach and is even lower than the naïve approach.
- (C3): The advantage of the order-based approach (CompOrder, BucOrder) is more pronounced when the delay time is short compared to the group-based approaches (Discontin_pp, Contin_reduce, Discontin_reduce, PeGaSus_delay).
- (C4): The data sensitivity truncation algorithm has an improvement for all methods (except BucOrder) on all datasets, especially on streams with large data domains such as Outpatient and Foodmart datasets.

D. Evaluation

In this section, we provide a detailed step-by-step verification of the experimental results included in the paper. (E1) and (E2) involve validating the effectiveness of the approaches proposed in this paper. (E3) and (E4) evaluate the impact of parameters on the proposed approaches, (E5) compares the proposed methods with state-of-the-art methods from other privacy settings.

1) *Experiment (E1)*: [The Accuracy Improvement of the Delay-allowed Approaches.][About 21min]

This part of the experiment corresponds to Figures 2 and Figure 3 in the paper. The approximate running times for the four datasets under two different delay_time parameters are shown in Table I.

TABLE I
THE RUNNING TIME OF EXPERIMENT (E1)

Delay	COVID	Unemployment	Outpatient	Foodmart
$w = 10$	5.9s	5.7s	1min33s	2min42s
$w = 100$	8.9s	22.5s	3min10s	13min12s

[Execution] There are two steps for verifying Figure 2 and Figure 3.

Step 1. Set ‘delay_time=10’ and run ‘python est_compall.py’. The terminal will sequentially display the error results of all methods on the four datasets, and comparison line charts for all methods will pop up. Please manually close the popped-up line charts for the code to continue running the next dataset.

Step 2. Set ‘delay_time=100’ and rerun ‘python est_compall.py’. Other operations are the same as in Step 1.

[Results] In each step, the results displayed in the terminal and the line charts popped up by the code correspond to Figures 2/Figure 3 (a), (b), (c), and (d) in the paper. There may be some fluctuations, the trend of each method’s error with changes in ϵ , and the relative magnitudes should be consistent with those in the paper. The observed results in this part verify C1, C2, and C3 in the Major Claims.

2) *Experiment (E2)*: [The Effectiveness of Data Sensitivity Truncation Algorithm.][About 7min47s]

This part corresponds to Figure 4 in the paper and is used to evaluate the effectiveness of the sensitivity reduction method proposed in this paper.

[Execution] Run ‘python est_sensitivity.py’.

[Results] The terminal will sequentially display the results of the Naive, Contin, Discontin, CompOrder, and BucOrder methods on the four datasets, both without sensitivity reduction and with sensitivity reduction. The data under “No Sensitivity Reduce” correspond to the red bars in Figure 4, while the results under “Sensitivity Reduce” correspond to the blue bars. Although there may be fluctuations due to randomness, the magnitude of the data should be consistent with those in Figure 4. For all datasets, except for the BucOrder method, the data under “Sensitivity Reduce” should show a decrease compared to “No Sensitivity Reduce”. The observed results in this part verify C4 in the Major Claims.

3) *Experiment (E3)*: [Comparison of Group-based Approaches and Order-based Approaches.][About 58s]

This part corresponds to Figure 5 in the paper, illustrating the impact of delay time length on group-based and order-based methods respectively.

[Execution] Run ‘python est_delaylength.py’.

[Results] In the terminal, the results for the discontinuous, CompOrder, and BucOrder methods on two datasets will be sequentially displayed, showing the error in published results as the parameter w varies from 10 to 90. The results for each dataset will be between “Results” and “Results End”. All data should correspond to the bars in Figure 5.

It should be observed from the results: a. The performance of the discontinuous approach improves as the length of the delay time increases. b. The order-based (CompOrder and BucOrder) approaches do not exhibit a clear pattern of accuracy variation with changes in the delay time. c. When the delay time is short, order-based (CompOrder and BucOrder) approaches demonstrate an advantage over group-based (discontinuous) approaches.

4) *Experiment (E4)*: [Impact of the key parameters.][About 18s and 12s]

This part of the experiment corresponds to Figures 6, 7, 8, 9, and 10, primarily evaluating the impact of parameter variations on the performance of the proposed algorithms.

[*Execution*] There are two steps in this part. We have already set all the parameters.

Step 1. Run ‘python est_order.py’ and manually close the popped-up line charts to proceed with running the next dataset.

Step 2. Run ‘python est_group.py’.

[*Results*] For Step 1, line charts similar to those in Figure 6 from the paper will pop up. It should be observed that too-small bucket size m can lead to a decrease in the accuracy of the released results.

For Step 2, the terminal will sequentially display the accuracy of the publication results for four group-based methods across different values of τ on two datasets. The results will show that the impact of θ on group-based methods is not monotonical. The magnitudes of the error results should correspond to the respective figures in the paper.

5) *Experiment (E5)*: [Compare with existing works come from other privacy settings.][DPI: 31s, Adapub: 4s] This part of the experiment corresponds to Table III in the paper. The source code for DPI and Adapub comes from their corresponding open-source libraries. We only made some minor modifications to make them meet the event-level privacy and set the parameters for comparison in our experiment.

[*Execution*] There are two steps in this part.

Step 1. Run ‘python est_other.py’.

Step 2. Run ‘python DPI_DEMO_script.py’.

[*Results*] In Step 1 of E1, it will show the error results of $\epsilon \in [0.1, 1]$ for Naïve, Discontin, CompOrder, and BucOrder on the Unemployment and Outpatient datasets. By dividing the error results (MAE) of all methods by the error results of naïve, the correctness of Table III can be verified. The proposed algorithm should show accuracy advantages over state-of-the-art solutions. Directly comparing the error results of these methods can also validate this.

E. Copyright

We have migrated the code repository from GitHub to Zenodo. The DOI for the repository is “DOI: 10.5281/zenodo.13643225”.

F. Acknowledgement

We would like to express our sincere gratitude to the AE reviewers for their thorough evaluation of our work. Your valuable feedback and insightful comments have significantly

contributed to improving both the quality of our code and the clarity of our paper.